Studio Nx

JS 07 — requireLogin()

2026-10-12

JS 07 — `requireLogin()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	06 `isLoggedIn()` · 08 `logout()`

1. 하는 일

로그인이 필요한 페이지의 맨 처음에 부르는 함수입니다. 로그인 상태면 `true`를 돌려주고, 아니면 로그인 화면으로 보낸 뒤 `false`를 돌려줍니다.

2. 언제 쓰나

로그인한 사람만 볼 수 있는 페이지 네 곳이 페이지 코드의 맨 처음에 이 함수를 부릅니다.

페이지	부르는 곳
마이페이지 `myPage.html`	`pages/mypage.js` (P1-17)
내 프로젝트 `myProject.html`	`pages/myproject.js` (P1-14)
회원 정보 `userInfo.html`	`pages/userinfo.js` (P1-10)
시퀀서 `studioNx.html`	`pages/sequencer.js`의 `init()`

```
const ok = await requireLogin();
if (!ok) return;          // 로그인 화면으로 이동 중 — 아래 코드(대시보드 불러오기 등)를 실행하지 않음
```

실제로 일어나는 일 (예)

로그아웃한 상태에서 주소창에 `localhost:8000/myProject.html#new`를 직접 입력하면 다음과 같이 진행됩니다.

순서	화면과 주소창
① 내 프로젝트 페이지가 열리기 시작합니다	`/myProject.html#new`
② `requireLogin()`이 로그인하지 않은 상태임을 확인합니다	—
③ 홈으로 이동하고 로그인 모달이 열립니다	`home.html#login?next=%2FmyProject.html%23new`
④ 로그인에 성공합니다 (P1-07)	—
⑤ 원래 가려던 곳으로 돌아오고, 새 프로젝트 모달까지 열립니다	`/myProject.html#new`

로그인한 상태라면 ②에서 `true`가 나오고 페이지가 그대로 이어집니다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 직접 쓰지 않습니다.** `isLoggedIn`과 `loginUrl`은 같은 파일 안에 있고, 그 함수들이 `api.js`를 씁니다.

함수

```
export async function requireLogin() {
  if (await isLoggedIn()) return true;
  location.replace(loginUrl(location.pathname + location.search + location.hash));
  return false;
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`await isLoggedIn()`	로그인 상태 확인이 끝날 때까지 기다린 뒤 `true` 또는 `false`를 받습니다 (06)
`location.pathname`	주소의 경로입니다. 예: `/myProject.html`
`location.search`	`?`부터 시작하는 부분입니다. 예: `?projectId=3` (없으면 `''`)
`location.hash`	`#`부터 시작하는 부분입니다. 예: `#new` (없으면 `''`)
`location.replace(url)`	`url`로 이동하면서 **지금 페이지를 방문 기록에서 지웁니다**

4. 흐름

1. `isLoggedIn()`이 `true`이면 바로 `true`를 돌려주고, 페이지는 계속 진행됩니다.
2. 아니면 지금 주소(경로, 쿼리, 해시)로 로그인 주소를 만듭니다 (04 `loginUrl()`).
3. 그 주소로 `location.replace`하고 `false`를 돌려줍니다.

5. 왜 이렇게

선택	이유
`location.href` 대신 `location.replace`	`href`로 이동하면 로그인 화면에서 뒤로 가기를 눌렀을 때 마이페이지로 돌아갔다가 **다시 로그인 화면으로 튕기는 일이 반복됩니다**. `replace`는 마이페이지를 방문 기록에서 지우므로, 뒤로 가기를 누르면 그 전 페이지로 갑니다
쿼리와 해시까지 넣음	`?projectId=3`(어떤 프로젝트인지)이나 `#new`(새 프로젝트 모달)가 빠지면 로그인한 뒤 엉뚱한 화면이 나옵니다. 원래 템플릿은 경로와 쿼리만 넣었는데, 해시도 더했습니다
`false`를 돌려줌	이동은 바로 끝나지 않습니다. 그사이 페이지 코드가 API를 부르지 않도록 `if (!ok) return;`으로 멈추게 합니다
판단은 `isLoggedIn()`에 맡김	토큰 처리와 재발급 처리를 다시 쓰지 않아도 됩니다

6. 시험 (fetch와 location을 가짜로 바꿔서 시험)

경우	결과
토큰 있음	`true` 이고, 이동하지 않습니다
토큰은 없고 리프레시 쿠키가 유효함	`true` 이고, 이동하지 않습니다
로그인하지 않은 상태로 `/myPage.html`	`false` 이고, `home.html#login?next=%2FmyPage.html` 로 이동합니다
`/studioNx.html?projectId=3`	`…next=%2FstudioNx.html%3FprojectId%3D3`
`/myProject.html#new`	`…next=%2FmyProject.html%23new`
위의 주소를 `parseLoginHash()` 로 다시 읽기	`/myProject.html#new` 가 그대로 나옵니다